

Methods of Primitives – Simple Hinglish Summary

Source: <https://javascript.info/primitives-methods>

Main baat

JavaScript primitives (strings, numbers, etc.) ke saath **aise kaam karne deta hai jaise wo objects hon**, aur unke methods bhi call kar sakte hain. Lekin asal me **primitives objects nahi hote**. Ye chapter batata hai ki ye kaise hota hai.

1. Primitive vs Object

Primitive:

- Primitive type ki value hai.
- 7 primitive types hain: `string`, `number`, `bigint`, `boolean`, `symbol`, `null`, `undefined`.

Object:

- Kai values ko properties ki tarah store kar sakta hai.
- `{}` se ban sakta hai, jaise `{name: "John", age: 30}`. JS me aur bhi objects hain, jaise **functions bhi objects** hain.

Objects ki ek badi khoobi: property me **function** store kar sakte hain.

```
javascript

let john = {
  name: "John",
  sayHi: function() {
    alert("Hi buddy!");
  }
};

john.sayHi(); // Hi buddy!
```

Kai built-in objects pehle se hain (dates, errors, HTML elements, etc.).

Lekin in features ki ek keemat hai: objects primitives se "**bhaari**" hote hain. Unhe internal machinery ke liye extra resources chahiye.

2. Primitive ko object ki tarah use karna

JavaScript ke creator ke saamne ek **paradox** tha:

- Primitive (jaise string ya number) ke saath bahut kuch karna chahte hain. Methods se access karna achha hoga.
- Lekin primitives **jित्ने ho sake tez aur halke** hone chahiye.

Solution thoda ajeeb hai, lekin ye hai:

1. Primitives **primitive hi rehte hain**. Ek single value, jaisa chahiye tha.
2. Language strings, numbers, booleans aur symbols ke **methods aur properties access** karne deti hai.
3. Ye kaam karne ke liye ek special "**object wrapper**" banta hai jo extra functionality deta hai, aur phir **destroy** ho jata hai.

Ye "object wrappers" har primitive type ke liye alag hain: `String`, `Number`, `Boolean`, `Symbol`, `BigInt`. Isliye unke methods ke sets alag hain.

Jaise string method `str.toUpperCase()` capitalized string return karta hai:

```
javascript  
  
let str = "Hello";  
  
alert( str.toUpperCase() ); // HELLO
```

`str.toUpperCase()` me asal me kya hota hai:

1. `str` ek primitive string hai. Uski property access karte hi ek **special object** banta hai jo string ki value jaanta hai aur jiske paas `toUpperCase()` jaise useful methods hain.
2. Wo method chalta hai aur nayi string return karta hai (jo `alert` dikhata hai).
3. Special object **destroy** ho jata hai, aur sirf primitive `str` bacha rehta hai.

Isliye primitives methods de sakte hain aur phir bhi **halke** rehte hain.

JS engine is process ko bahut optimize karta hai. Wo extra object banana **skip bhi kar sakta hai**, lekin specification ke hisaab se aise behave karna zaruri hai jaise object bana ho.

Number ke bhi apne methods hain, jaise `toFixed(n)` number ko given precision tak round karta hai:

```
javascript  
  
let n = 1.23456;  
  
alert( n.toFixed(2) ); // 1.23
```

(Aur methods aage Numbers aur Strings chapters me hain.)

3. `String/Number/Boolean` constructors sirf internal use ke liye

Java jaisi languages me `new Number(1)` ya `new Boolean(false)` se explicitly wrapper objects bana sakte hain. JS me historical reasons se ye possible hai, lekin **bilkul recommend nahi**. Kai jagah cheezein bigad jayengi.

javascript

```
alert( typeof 0 ); // "number"
```

```
alert( typeof new Number(0) ); // "object"!
```

Objects (`if`) me hamesha truthy hote hain, isliye ye alert dikhega:

javascript

```
let zero = new Number(0);
```

```
if (zero) { // zero true hai, kyunki ye object hai
  alert( "zero is truthy!?" );
}
```

Lekin (`new`) ke bina wahi functions (`String/Number/Boolean`) use karna **bilkul theek aur kaam ka** hai. Ye value ko corresponding type (primitive) me convert karte hain:

javascript

```
let num = Number("123"); // string ko number me convert kiya
```

4. (`null`) / (`undefined`) ke methods nahi hote

Special primitives (`null`) aur (`undefined`) exceptions hain. Inke **wrapper objects** nahi hain aur koi methods nahi. Ek tarah se ye "**sabse zyada primitive**" hain.

Inki property access karne par error aata hai:

javascript

```
alert(null.test); // error
```

Summary

- (`null`) aur (`undefined`) ko **chhodkar** baaki primitives bahut helpful methods dete hain.
- Formally ye methods **temporary objects** ke through kaam karte hain, lekin JS engines is process ko andar se bahut optimize karte hain, isliye inhe call karna **mehanga nahi** hai.

Practice Task (Answer)

Kya string me property add kar sakte hain?

```
javascript

let str = "Hello";

str.test = 5;

alert(str.test);
```

Jawab: `use strict` ho ya na ho, is par depend karta hai:

1. **Non-strict mode me:** `undefined`
2. **Strict mode me:** error

Kyu? Line `str.test = 5` par kya hota hai:

1. `str` ki property access hote hi ek **"wrapper object"** banta hai.
2. **Strict mode me** us par likhna error hai.
3. Warna operation chalta hai, object me `test` property ban jati hai, lekin uske baad **wrapper object gayab** ho jata hai. Isliye last line me `str` me us property ka koi nishan nahi hota.

Ye example saaf dikhata hai ki primitives objects nahi hain. Wo extra data store nahi kar sakte.

Quick Summary

Baat	Yaad rakho
Primitive	Ek single value, halka
Object	Kai values store kar sakta hai, bhaari
Primitive ke methods	Temporary wrapper object ke through chalte hain
Wrapper types	<code>String</code> , <code>Number</code> , <code>Boolean</code> , <code>Symbol</code> , <code>BigInt</code>
<code>new Number(0)</code>	Kabhi mat use karo (object banta hai, <code>typeof</code> = <code>"object"</code> , truthy hai)
<code>Number("123")</code>	Theek hai (<code>new</code> ke bina, primitive me convert karta hai)
<code>null</code> / <code>undefined</code>	Koi methods nahi, property access = error
<code>str.test = 5</code>	Primitive extra data store nahi kar sakta